

1. Explain Programming and python in detail.

Definition of Programming:-

Programming is the process of writing a code that a computer can understand and execute to perform specific tasks or to solve problems.

Purpose of Programming:-

- The main purpose of programming are:-
- To solve real-world problems using computer.
- To process and analyse data efficiently.
- To build software applications, websites, games and many more.

Definition of Python:-

Python is a high-level, interpreted, programming language. created by Guido van Rossum and released in 1991. It is known

for its simple syntax and readability.

Characteristics of Python:-

→ Simple and easy to learn:-

Python uses simple syntax, which makes it beginner friendly.

→ High level language:-

Python handles memory management automatically, allowing programmers to focus on logic rather than hardware details.

→ Interpreted language:-

Python code is executed line by line, which makes debugging easier.

→ Dynamically typed:-

Variables do not require explicit data type declaration.

→ Object oriented:-

Python supports object-oriented programming concepts like classes and objects.

Applications of python:-

- Web development.
- Data Science and Data Analysis.
- Artificial Intelligence.
- Machine learning.

- Game development.
- Cybersecurity.

Comments:-

Comments are used to explain code, make it more readable, and help programmers to understand the logic.

1. Single-line Comments:-

Used to write comments on a single line.

Syntax:- # This is a single-line comment.

2. Multi-line Comments:-

Python does not have a specific multi-line comment symbol. but we can use.

Syntax:- # This is line one

This is line two.

This is line three.

Importance in Python in Modern Software development:-

→ High demand in Industry:-

Python is widely used by companies.

like Google, Facebook, Netflix, and NASA.

2. Supports Emerging Technologies:-

Python is a leading language like, in AI, Machine learning, Data Science, and Automation.

3. Rapid Development:-

Python allows faster development with fewer lines of code compared to other languages.

4. Versatility:-

Python can be used for small scripts as well as large-scale enterprise applications.

2. Describe Data types and Operators in python with suitable examples.

Data types in Python:-

A data type specifies the type of data a variable can store. Python provides built-in data types to handle different kinds of data.

1. Built-in Data types in Python:-

→ Numeric Data types:-

Used to store numerical values.

Examples:- int, float, complex.

```
a = 10
```

```
b = 2.5
```

→ Sequence Data Types:-

Used to store ordered collections.

Example:- str, list, tuple.

```
name = "python"
```

```
nums = [1, 2, 3]
```

→ Set Data Type:-

Stores unordered and unique elements.

```
s = {1, 2, 3}
```

→ Mapping Data Type:-

Stores data types in key-value pairs.

```
student = {"name": "Alex", "age": 20}
```

→ Boolean Data Type:-

Stores True or False.

```
result = True
```

Type Identification using type()

The type() function is used to identify the data type of variable.

```
x = 5  
type(x) #int
```

Operators in Python:-

Operators perform operations on variables and values.

1. Arithmetic Operators:-

Used for mathematical calculations (+, -, *, /, %)

Example:- $a = 10 + 5$

2. Assignment Operators:-

Used to assign and update values.

Operator

Example

Meaning

=

$x = 5$

$x = 5$

$+=$

$x += 2$

$x = x + 2$

$-=$

$x -= 2$

$x = x - 2$

$*=$

$x *= 2$

$x = x * 2$

$/=$

$x /= 2$

$x = x / 2$

3. Comparison Operators:-

Used to compare values and return

True or false.

Operators:- $==$, $!=$, $>$, $<$, $>=$, $<=$

Example:-

$x = 10$

$y = 5$

print ($x > y$)

4. Logical Operators:-

Used to combine conditional statements.

Operator
and

Meaning

True if both conditions true.

or

True if atleast one condition is true.

not

Reverse the result.

Example:-

$a = 10$

print ($a > 5$ and $a < 20$)

5. Membership Operators:-

Used to check whether a value exists in a sequence.

Operator

Meaning

in

Value exists.

not in

Value does not exist.

Example:-

```
colors = ["red", "blue"]  
print("red" in colors)
```

6. Identity Operator:-

Used to compare memory locations.

Operator

is

is not

Meaning

Same object

Different object

Example:-

```
x = 10
```

```
y = 10
```

```
print (x is y)
```

Real-World Usage of operators:-

1. Arithmetic Operators:- Used in banking apps to calculate interest, salary or bills.
2. Comparison Operators:- Used in exams systems to check pass / fail conditions.
3. Logical Operators:- Used in logic systems for username and password verification.
4. Assignment Operators:- Used to update values in programs.

5. Membership Operator:-

Used in search engines to find keywords

6. Identity Operator:-

Used in memory management and object comparison.

3. Explain Python Input and Output operations in detail.

1. input() function:-

The input() function is used to accept input from the user through the keyboard.

Syntax:- variable = input("Message")

→ The default data type of data returned by input() is string (str), even if the user enters a number.

example:-

```
age = input("Enter your age:")
```

```
print(age)
```

Type Conversion while taking Input:-

Since input() returns a string, type conver-

• sion is needed to perform numerical operations.

Common Type Conversions:-

- `int()` - converts to integer.
- `float()` - converts to decimal number.
- `str()` - converts to string.

Example:-

```
age = int(input("Enter your age: "))
```

```
salary = float(input("Enter salary: "))
```

Taking Multiple Inputs:-

1. Using Separate `input()` statements:-

```
a = int(input())
```

```
b = int(input())
```

Taking Multiple inputs in one line using `split()`:

```
a, b = input().split()
```

With type conversion:

```
a, b = map(int, input().split())
```

Output in Python (`print()` Function)

The `print()` function is used to display output on the screen.

Syntax:-

```
print (value1, value2, sep=' ', end='\n')
```

- Separators (sep)

→ Specifies how multiple values are separated.

Examples:-

```
print ("Python", "Java", sep=", ")
```

- Formatted Output (format specifiers):

→ Using % format specifiers:

%d → integer

%f → float

%s → string

Example:-

```
print ("Age: %d Marks: %.2f" % (age, marks))
```

→ Using format() method:-

Example:-

```
print ("Name: {} Age: {}".format(name, age))
```

→ Using f-strings:-

Example:-

```
print (f"Name: {name}, Age: {age}")
```


4. Discuss Control Statements and Decision Making statements in Python.

Meaning and Importance of Control Statements.

- Control statements control the flow of execution of a program.
- Enable decision-making based on conditions.
- Allow repetition of tasks using loops.
- Improve readability and structure of code.

Types of Control Statements in Python:-

→ Decision-making Statements:-

Used to choose actions based on conditions.

Examples: if, if-else, if-elif-else

→ Looping statements:-

Used to repeat a block of code.

Examples: for, while.

→ Jump statements:-

Used to change the flow inside loops.

Example:- break, continue ; p.

Decision-Making Statements:-

→ if statement:-

Executes code only when the condition is true.

Syntax:-

```
if condition:  
    statement(s)
```

Example:-

```
if x > 0:  
    print("Positive number")
```

→ if-else statement:-

Executes one block if condition is true, otherwise executes another block.

Syntax:-

```
if condition:  
    statement(s)
```

```
else:  
    statement(s)
```

Example:-

```
if x % 2 == 0:
```

```
    print("Even")
```

```
else:
```

```
    print("Odd")
```

→ if - elif - else statement :-

Used to check multiple conditions.

Syntax:- if condition1:

statement(s)

elif condition2:

statement(s)

else:

statement(s)

Example:- if marks >= 90:

print("Grade A")

elif marks >= 60:

print("Grade B")

else:

print("Fail")

Syntax flow and execution Control:-

→ Conditions are evaluated from top to bottom.

→ Only the block with a true condition is executed.

→ Python uses indentation to define code blocks.

→ If no condition is true, else block is executed.

5. Write an essay on Python Programming fundamentals.

Role of Programming in Problem solving:-

→ Programming helps solve real-world problems by breaking them into small logical steps.

→ It enables automation of tasks, decision making, and calculation of results based on given data.

→ A program takes input, processes it using logic, and produces output to meet a desired goal.

Python Syntax Simplicity and Readability

→ Python's syntax is simple, clean, and easy to understand, similar to written English.

→ It uses indentation instead of braces to define blocks of code, improving readability.

→ This simplicity makes Python suitable for beginners and helps in writing concise and maintainable code.

Use of comments for code

Documentation.

→ Comments explain code and are ignored by the interpreter during execution.

→ They improve readability and help others (and yourself later) understand the purpose of code.

→ In Python.

- Single-line comments starts with `#`
- Multi-line comments can use triple quotes (`""" ... """`)

Data Types, Operators and Input/Output Operations.

→ Python supports various datatypes

such as integers, float, strings, booleans, lists, tuples, dictionaries, etc.

→ Operations perform actions on data:

- Arithmetic (+, -, *, /)
- Comparison (==, !=, >, <)
- Logical (and, or, not)

→ Input/output Operations:

- `input()` is used to take input from the user and returns a string by default.
- `print()` displays output on the screen.
- Type conversion (eg., `int()`, `float()`) is used when numeric input is needed instead of strings.

Control flow Using Decision-Making Statement:-

→ Control flow statements decide which

part of the program runs based on conditions.

→ Decision-making statements in Python include:-

- if - executes a block only if condition is true.
- if - else - executes one block if true, otherwise another.
- if - elif - else - checks multiple conditions in sequence.

→ These statements help the program make logical decisions and choose appropriate actions based on data.

Solve below Real-world problems using

Python Programming:-

1. Movie Ticket Pricing

A movie theatre charges:

₹150 for children (age < 13)

₹250 for adults (age 13-59)

₹200 for seniors (age ≥ 60)

if the person is watching a 3D movie, add ₹50 extra.

Write a program that takes age and is 3D (1 or 0) and prints the final ticket price.

```
age = int(input("Enter your age:"))
```

```
3D = int(input("Enter (0, 1)"))
```

```
if age > 13:
```

```
    print("price=150")
```

```
elif age <= 59:
```

```
    print("price=250")
```

```
else:
```

```
    print("price=200")
```

```
if 3D == 1:
```

```
    price += 50
```

print (price)

2. College Attendance Rule

A college student is allowed to write the exam if:

attendance $\geq 75\%$

OR

attendance ≥ 60 And has medical certifi-

-cate (1=yes, 0=no) Take attendance percentage and medical certificate as input and print "Allowed" or "Not Allowed".

```
attendance = float(input("Enter attendance: "))
```

```
medical = int(input("Have Medical certificate (0,1): "))
```

```
if attendance  $\geq 75$  or (attendance  $\geq 60$  and medical == 1):
```

```
    print("Allowed")
```

else:

```
    print("not Allowed")
```

3. E-commerce Discount

A shopping site gives:

20% discount if bill ≥ 500

10% discount if bill is between 2000
and 4999.

No discount if bill < 2000

But if the customer is prime member,
they get extra 5% discount.

Input: bill amount, is Prime (1 or 0)

print final amount to be paid.

```
bill = int(input(" Bill amount: "))
```

```
prime = int(input(" prime member? (1/0): "))
```

```
discount = 0
```

```
if bill  $\geq 5000$ :
```

```
    discount = 0.20
```

```
elif bill  $\geq 2000$ :
```

```
    discount = 0.10
```

```
if prime == 1:
```

```
    discount += 0.05
```

```
final = bill * (1 - discount)
```

```
print(final)
```

4. Smartphone Battery Warning

A phone shows:

"Low Battery" if $\text{battery} \leq 20$

"Normal" if battery between 21-80

"Full" if $\text{battery} > 80$

But if phone is charging, it should show "charging" instead of any message.

Input: battery percentage, is charging (1 or 0)

```
battery = int(input("Enter battery percentage: "))
```

```
charging = int(input("charging (0/1): "))
```

```
if charging == 1:
```

```
    print("Charging")
```

```
elif battery <= 20:
```

```
    print("Low Battery")
```

```
elif battery <= 80:
```

```
    print("Normal")
```

```
else:
```

```
    print("Full")
```

5. Driving License check

A person can get a driving license if:

age ≥ 18
AND passed driving test (1-yes)
But if age ≥ 60 , driving test is not required.

Input: age, test Passed.

Print "Eligible" or "Not Eligible".

age = int(input("Enter age: "))

passed = int(input("Test passed (1/0): "))

if age ≥ 60 or (age ≥ 18 and passed == 1):
 print("Eligible")

else:
 print("Not Eligible")

6. Online food Delivery

A restaurant gives free delivery if:

order amount ≥ 500

OR

user is a gold member

But if the distance is more than 10km,
delivery is never free.

Input: amount, is Gold (1 or 0), distance.

print "Free Delivery" or "Delivery charged".

amount = int(input("Enter order amount: "))

Goldmember = int(input("Gold member (1/0): "))

```
distance = int(input("Enter distance (km): "))
```

```
if distance > 10:
```

```
    print("Delivery charged")
```

```
elif amount >= 500 or Goldmember == 1:
```

```
    print("Free Delivery")
```

```
else:
```

```
    print("Delivery charged")
```

7. Bank loan Approval

A bank approves a loan if: salary $\geq$ 30,000 AND credit score $\geq$ 700 OR salary $\geq$ 50,000 (credit score ignored)

Input: salary, credit score

print "Loan Approved" or "Loan Rejected".

```
sal = int(input("Enter salary"))
```

```
creditscore = int(input("Enter credit score: "))
```

```
if (sal >= 30000 and creditscore >= 700) or sal >= 50000:
```

```
    print("Loan Approved")
```

```
else:
```

```
    print("Loan Rejected")
```

8. Electricity Bill

Units consumed:

First 100 units $\rightarrow$ ₹2/unit

Next 100 units $\rightarrow ₹3/\text{unit}$
Above 200 units $\rightarrow ₹5/\text{unit}$

Note: No loops

print final bill amount

```
units = int(input("Enter units: "))
```

```
if units <= 100:
```

```
    bill = units * 2
```

```
elif units <= 200:
```

```
    bill = (100 * 2) + (units - 100) * 3
```

```
else:
```

```
    bill = (100 * 2) + (100 * 3) + (units - 200) * 5
```

```
print("Electricity Bill: ₹", bill)
```

9. Student Scholarship

A student gets a scholarship if:

marks ≥ 85

AND

family income < 500000

But if the student is a single parent child, income condition is ignored.

Input: marks, income, single parent (1 or 0)

```
marks = int(input("Enter marks: "))
```

```
income = int(input("Enter family income: "))
```

```
singleparent = int(input("Single parent (1/0): "))
```

```
if marks >= 85 and (income < 500000 or  
    singleparent == 1):
```

else:

print("Not Eligible")

10. Online Exam Result

A student passes if:

-theory ≥ 40 AND practical ≥ 40

But if total (theory + practical) ≥ 100 , pass even if one is less than 40.

Input: theory, practical

-theory = int(input("Enter theory marks:"))

practical = int(input("Enter practical marks:"))

if (-theory ≥ 40 and practical ≥ 40) or (theory + practical ≥ 100):

print("pass")

else:

print("fail")

11. Hotel Room Pricing

A hotel charges:

₹ 3000 per day for normal days

₹ 4000 per day on weekends

If customer stays more than 3 days, give

15% discount.

Input: is weekend (1 or 0), daysStayed

print final bill

Weekend = int(input("Is weekend? (1=yes, 0=no): "))
days = int(input("Days stayed: "))

if weekend == 1:

rate = 4000

else:

rate = 3000

total = rate * days

if days > 3:

total *= 0.85

print("Final Hotel Bill: ", total)

12. Gaming Level Unlock

A game unlocks next level if:

score $\geq$ 100

OR

player has a premium pass

But if player used cheating, access is denied.

Input: score, isPremium, used cheat

score = int(input("Enter score: "))

ispremium = int(input("Premium pass (0/1): "))

usedcheat = int(input("Used cheat (0/1): "))

if usedcheat == 1:

print("Access Denied")

```

elif score >= 100 or ispremium == 1:
    print("Next level unlocked")
else:
    print("Level locked")

```

13. Mobil Data Usage

A network gives unlimited data if:
 daily usage $\leq$ 2GB

OR
 user has unlimited plan
 But if roaming is on, unlimited plan
 does not work.

Input: dataUsed, has Unlimited plan, isRoaming.

```

dataUsed = float(input("data used (GB): "))
plan = int(input("Unlimited plan (0/1): "))
isRoaming = int(input("Roaming on? (0/1): "))
if isRoaming == 1:

```

```

    print("limited data")

```

```

elif dataUsed <= 2 or has plan == 1:
    print("Unlimited data")

```

```

else:
    print("limited data")

```

14. Office Entry System

An employee can enter the office if:

ID card is valid
AND (fingerprint matches OR face scan matches)
But if it is a holiday, entry is denied
for everyone.

Input: id valid, fingerprint, facescan, is-holiday.

```
idvalid = int(input("ID valid (0/1): "))  
fingerprint = int(input("Finger print match (0/1): "))  
facescan = int(input("facescan match (0/1): "))  
holiday = int(input("is holiday (0/1): "))  
  
if holiday == 1:  
    print("Entry Denied")  
elif idvalid == 1 and (fingerprint == 1 or facescan == 1):  
    print("Entry Allowed")  
else:  
    print("Entry Denied")
```

15. Movie Rating Display

A movie app shows rating band on
average score:

Average $\geq 8.5 \rightarrow$ "Excellent"

Average between 6.0 and 8.4 $\rightarrow$ "Good"

Average $< 6.0 \rightarrow$ "Average"

But if the movie is marked as editor's choice, always show "Recommended".

Input: averageRating, isEditorsChoice (1 or 0)

print the message.

```
rating = float(input("Enter average rating: "))  
isEditorsChoice = int(input("Editor's choice (1=yes, 0=no): "))
```

```
if isEditorsChoice == 1:
```

```
    print("Recommended")
```

```
elif rating >= 8.5:
```

```
    print("Excellent")
```

```
elif rating >= 6.0:
```

```
    print("Good")
```

```
else:
```

```
    print("Average")
```